

The Markdown File Guide

How to write structured documents that humans and computers can both read.

by d.avebrown

Markdown is a plain-text document format that is easy for humans to write and easy for computers, websites, documentation tools, and AI assistants to understand.

The point is not to "learn a technical file format." The point is: Markdown lets you create clean, portable, structured documents that any tool can read without getting confused by formatting junk.

Here is everything you need to know — from basic syntax to advanced techniques for working with AI tools, version control, and documentation pipelines.

FOUNDATIONS

Three core concepts to understand first

01 — What Is Markdown?

Markdown files usually end in `.md`. They are plain text files with simple formatting symbols. Instead of using a visual editor like Google Docs or Word, Markdown uses lightweight syntax to show structure.

The `#` means title. The `##` means section heading. Blank lines separate ideas. The document has hierarchy. That sounds basic, but it matters a lot when you are asking an AI to understand a document, summarize research, or follow instructions — because the structure is explicit and readable by both people and machines.

EXAMPLE

```
# Document Title ## Background This is the context for what this document is about. ## Goal What this document is trying to accomplish. ## Notes Anything important the reader should keep in mind.
```

02 — What Is a CLAUDE.md File?

A CLAUDE.md file is an instruction or context file for Claude. Think of it like a read-me-first note for a folder. When Claude opens a project directory, it reads this file first to understand what the project is, what files matter, and what it should know before helping.

A good CLAUDE.md includes: a project summary, a description of the audience or purpose, a list of important files and what they contain, the tone or style to use, any constraints or approved content, and specific instructions for how Claude should behave in that folder. It gives Claude a map of the room before it starts rearranging furniture.

EXAMPLE

```
# Project Context This folder contains materials for [project name]. ## Important Files -  
`research.md`: background research and source notes - `draft.md`: current working draft ##  
Instructions Refer to research notes before drafting. Keep tone consistent with existing drafts.
```

03 — How to Start a Markdown Document

The opening of a Markdown file sets the context for everything that follows. AI tools often use the title, headings, and first few paragraphs to understand what kind of document they are looking at — so a strong opening matters.

A vague title like "Notes — Stuff from last week" tells nobody anything. A specific title like "Q3 Planning Session Notes — October 14: Decisions, Action Items, and Open Questions" tells both the reader and any AI tool exactly what the document is before reading a single line. Invest a few extra words in the title and opening paragraph — it pays off every time the document is opened.

PATTERN

```
# [Specific Title] One sentence describing what this document is and why it exists. ## Background  
## Goal ## Notes
```

SYNTAX & STRUCTURE

04 — Markdown Is Portable

Markdown is not locked inside one app. A .md file can be opened in a code editor, Notion, GitHub, Obsidian, many CMS systems, documentation tools, and AI assistants. That makes it useful for any content that may need to move between tools over time. You write the file once and it works almost everywhere, without conversion headaches or format lock-in.

05 — Markdown Keeps Formatting Clean

Markdown does not carry hidden formatting like Word or Google Docs sometimes do. That means fewer weird font issues, spacing problems, broken bullets, mystery styles, and copy-paste disasters. When you send Markdown content into an AI tool or share it across platforms, it arrives as clean text — no invisible baggage attached.

06 — Headings Create Structure

Headings are one of the most important parts of Markdown. They create visible hierarchy that both people and AI tools can navigate.

SYNTAX

Document Title ## Main Section ### Subsection

Use headings to separate distinct parts of a document: Background, Goals, Key Points, Open Questions, Next Steps. This structure helps people skim quickly and helps AI understand what each section is supposed to do.

07 — Bullets Are Great for Lists and Ideas

Markdown makes unordered lists simple. Prefix each line with a hyphen, asterisk, or plus sign. This is useful for brainstorming, reference lists, feature sets, research findings, and outlines. AI tools tend to handle structured bullets much better than giant unbroken paragraphs — bullets communicate clearly that each item is distinct.

SYNTAX

- Item one - Item two - Item three

08 — Numbered Lists Are Useful for Processes

Use numbered lists when order matters — workflows, step-by-step instructions, checklists, and standard operating procedures. The number signals to both the reader and the AI that sequence is significant. Numbered lists are also easier for AI to reference specifically: "In step 3, you said..." is clearer than "In that paragraph you mentioned..."

SYNTAX

1. First step 2. Second step 3. Third step

09 — Markdown Supports Links Cleanly

Links in Markdown use a readable format that keeps documents scannable. This is helpful when collecting references in a research document, a resource list, or a project brief. It keeps documents readable instead of cluttering them with raw URLs.

SYNTAX

[Link text](https://example.com) ## Reference Links - [Research paper](https://example.com) - [Source document](https://example.com)

10 — Emphasis: Bold and Italic

Markdown supports bold and italic text. Use emphasis sparingly — Markdown works best when structure does most of the work. A good rule: if everything is emphasized, nothing is. Reserve bold for genuinely critical points and italic for titles, technical terms, or light clarification.

SYNTAX

*****bold text** *italic text****

11 — Markdown Is Great for Reusable Instructions

Markdown is excellent for storing instructions and templates because it keeps them organized, readable, and easy to copy. A prompt or workflow stored as a Markdown file is far more useful than one buried in a chat thread or email — you can find it, update it, and reuse it consistently.

EXAMPLE

```
# Document Review Instructions ## Role You are a careful editor reviewing a draft for accuracy and clarity. ## Task Review the attached draft and identify any factual errors or unclear sections. ## Rules - Focus on accuracy first, style second - Flag any claims that need a source - Do not rewrite — only annotate
```

12 — Markdown Works Well with Version Control

Because Markdown is plain text, tools like Git can show exactly what changed between versions — which lines were added, removed, or modified, and by whom. This is much more precise than tracking changes in Word or comparing Google Doc history. It is particularly valuable for living documents like technical specifications, policy documents, research notes, or any file that gets updated regularly by multiple people.

13 — Markdown Reduces Ambiguity for AI

AI tools perform better when their input is clear and structured. Markdown helps because it creates obvious sections, hierarchy, and labels. A wall of unformatted text leaves the AI guessing what each part is. A structured Markdown document tells it exactly what the audience is, what the goal is, and what constraints apply — before it generates a single word.

STRUCTURED BEATS UNSTRUCTURED

```
# Request ## Audience Internal team members ## Goal Explain the new onboarding process ## Tone Professional and clear ## Key Points - The new process starts on [date] - All team members need to complete training by [date]
```

GOING FURTHER

14 — Code Blocks for Technical Content

Markdown supports code blocks for displaying technical content exactly as written, without reformatting. Inline code uses single backticks. Fenced code blocks use triple backticks and can optionally specify the language for syntax highlighting. This is essential for documentation, command references, configuration snippets, and any content where exact formatting is part of the meaning.

SYNTAX

Inline: ``code here`` Fenced block: ````javascript function example() { return true; } ````

15 — Blockquotes for Callouts and Citations

Blockquotes use the > character and work well for quoting source material, highlighting key insights, or distinguishing referenced text from your own writing. They are especially useful in research notes and reference documents where the source of an idea matters.

SYNTAX

> "The purpose of writing is to communicate, not to impress." — Source or attribution

16 — Tables for Structured Data

Markdown supports simple tables using pipes and dashes. Tables are useful for comparing options, tracking status across multiple items, or any information that has a clear row-and-column structure. Keep them simple — wide or deeply nested tables become hard to read in raw Markdown, even if they render cleanly.

SYNTAX

```
| Column 1 | Column 2 | Column 3 | |-----|-----|-----| | Data | Data | Data | | Data |  
Data | Data |
```

17 — Nested Lists for Hierarchy

Lists can be nested by indenting with spaces or tabs to show parent-child relationships. This is useful for outlines, documentation with categories and subcategories, or project structures. Use nesting sparingly — more than two or three levels deep becomes hard to read and is usually a sign the content should be reorganized rather than indented further.

SYNTAX

```
- Main item - Sub-item - Sub-item - Sub-sub-item - Main item
```

18 — YAML Front Matter for Metadata

Many Markdown tools support YAML front matter: a block of metadata at the very top of the file, wrapped in triple dashes. Front matter is useful for tracking document status, tagging files for search or filtering, and storing metadata that tools can read programmatically. Not every Markdown app supports it, but it is widely used in documentation systems, static site generators, and note-taking apps like Obsidian.

SYNTAX

```
--- title: Document Title author: Your Name date: 2025-01-01 tags: [reference, documentation] status: draft --- # Document Title
```

19 — Converting Markdown to Other Formats

Because Markdown is plain text, it converts cleanly to other formats using tools like Pandoc. You can export to HTML for web publishing, PDF for sharing or printing, DOCX for Word-compatible documents, or even slides for presentations. This makes Markdown a useful source format: write once, export to whatever format the situation requires. Many documentation platforms like MkDocs and Jekyll also convert Markdown automatically as part of their build process.

PANDOC COMMAND

```
pandoc input.md -o output.pdf
pandoc input.md -o output.docx
pandoc input.md -o output.html
```

20 — Markdown Editors and Tools

You do not need a special app to write Markdown — any text editor works. But the right tool makes it easier. Popular options include VS Code (free, with Markdown preview and extensions), Obsidian (note-taking built around Markdown files), Typora (minimal editor with live preview), iA Writer (clean writing environment), and Notion (visual tool that imports and exports Markdown). GitHub also renders Markdown automatically in repositories, making it a good place to store and share documentation.

21 — Task Lists and Checkboxes

GitHub-flavored Markdown and many Markdown apps support interactive task lists using brackets. These render as clickable checkboxes in tools that support them, including GitHub, Obsidian, and Notion. Task lists are especially useful for project checklists, review processes, step-by-step procedures, and action items from meetings.

SYNTAX

- [x] Completed task - [] Incomplete task - [] Another item to do

22 — File Naming Conventions

Good file naming makes a collection of Markdown files much easier to navigate and maintain. A folder of 20 well-named files is easy to understand at a glance. A folder of 20 files named `notes.md`, `doc1.md`, and `final_FINAL2.md` is not. Consistency matters more than any specific convention — pick one approach and use it everywhere.

CONVENTIONS

Use lowercase: meeting-notes.md Use hyphens: project-brief.md Include dates when relevant: 2025-01-15-kickoff.md Be descriptive: onboarding-checklist.md

23 — HTML Comments for Hidden Notes

Standard Markdown does not have a native comment syntax, but HTML comments work inside Markdown files. Comments are invisible in rendered output but visible in the raw file. This is useful for leaving notes to collaborators, temporarily hiding sections without deleting them, tracking document status inline, or leaving instructions for AI tools that you do not want to appear in the final output.

SYNTAX

```
<!-- This note is hidden in the rendered output --> <!-- TODO: verify this section before  
publishing --> <!-- AI: do not include this section in your summary -->
```

24 — Converting PDFs to Markdown

You can convert existing PDFs into Markdown using two tools: **poppler** (which extracts text from PDFs while preserving layout) and **pandoc** (which converts that text into clean Markdown). Works best on text-based PDFs — scanned image PDFs require OCR software first. The output may need light cleanup, but it gets you a workable, editable file far faster than retyping.

STEP 1 — INSTALL THE TOOLS (MAC)

```
brew install poppler  
brew install pandoc
```

STEP 2 — CONVERT

```
pdftotext -layout input.pdf - | pandoc -t markdown -o output.md
```

pdftotext -layout preserves the original column and spacing structure of the PDF. Piping it into **pandoc** then converts that plain text into structured Markdown. Once converted, the file is fully editable, searchable, and ready to feed into AI tools — far more useful than a locked PDF.

Want more practical AI guides?

Follow along on Instagram for weekly breakdowns.

@d.avebrown